多模态大模型原理与应用开题报告

题目: **网页多模态 Agent**

姓　　名	王胜伟，王一澄，吴乐超
学　　号	23336228，23336233，23336250
院　　系	计算机学院
专　　业	计算机科学与技术
指导教师	刘阳

2026 年 5 月 7 日

第 1 章　任务定义与预期效果

本学期的大作业，我们小组选择题目 7：**网页多模态 Agent**。

题目要求基于网页截图/结构信息和自然语言指令，生成一系列浏览器操作（点击、输入、滚动等）完成具体任务，并在浏览器自动化环境中演示。课程给出的**最低验收标准**包括：

- 选择 1–2 个真实网站，针对每个网站至少设计 3 个任务场景；
- 每个场景至少 10 次独立测试（共 $\geqslant 60$ 次），统计任务成功率；
- 分类记录失败类型（识别失败/规划失败/执行失败）；
- 提供可视化操作轨迹（截图序列 + 动作序列）及错误分析。

我们希望实现的效果是：用户输入一段自然语言任务 prompt（给定涉及的起始网站 URL），Agent 自动打开网页，读取当前页面截图、DOM 和可交互元素信息，判断下一步应该点击、输入、滚动、等待还是返回。每一步执行后，系统都会重新观察页面状态，检查动作是否有效推进任务；当任务完成时，Agent 输出最终结果，并保存完整的截图序列、动作序列和执行日志，最后也可以对 Agent 的结果进行批量评测、轨迹回放和失败归因等。

第 2 章　相关工作调研

2.1　网页 Agent 研究

网页 Agent 的研究正从静态网页问答走向**真实交互式任务**。Mind2Web[1] 提供了跨 137 个网站、31 个领域的网页操作数据集，强调真实网页 HTML 过长，需要先筛选候选元素再进行动作预测。WebArena[2] 构建了可自托管的真实感网页环境，使用端到端任务成功率衡量 Agent 能力。VisualWebArena[3] 在 WebArena 基础上加入视觉密集网页任务，说明仅依赖 DOM/文本不足以处理复杂视觉布局、图像和图标。WebVoyager[4] 面向真实在线网站，整理了 15 个网站、643 个任务，并采用截图、Accessibility Tree 和多模态模型完成网页操作，是本项目最直接的系统参考。BrowserGym/AgentLab[9] 则提供统一的 WebAgent 实验框架，适合参考其 observation/action space 和批量评测流程。

在方法层面，SeeAct[5] 重点研究多模态模型作为网页 Agent 时的 **grounding** 问题，即模型可能知道下一步要点击什么，但难以准确定位。Set-of-Marks[6] 通过在截图上叠加编号标记，让模型用编号引用视觉区域，从而降低坐标预测难度。ReAct[7] 框架将推理和行动交替进行，适合构建“观察–思考–动作–反馈”的多步任务循环。

2.2 通用 GUI Agent 的启发

OSWorld[10]、Windows Agent Arena[11] 和 AndroidWorld[12] 等工作把 Agent 从网页扩展到桌面操作系统与移动端应用，强调真实环境、任务初始化、状态检查和可复现实验。UI-TARS[13] 进一步探索端到端 GUI Agent，用截图输入直接预测鼠标键盘动作；OmniParser[14] 则把 UI 截图解析为结构化元素，和本项目的观察构建器、候选元素提取非常接近。SeeClick[15] 提出的 ScreenSpot 以及后续 ScreenSpot-Pro[16] 则说明 GUI grounding 是视觉 Agent 的关键瓶颈。这些工作超越了本次项目 WebAgent 的范畴，但提供了很重要的启发。我们可以把**可复现任务定义、执行式验证、完整轨迹记录和失败类型分析**等的思想应用到浏览器环境中，聚焦 WebAgent 上面。

表 2.1　代表性相关工作

方向	代表工作	对本项目的启发
网页任务数据集	Mind2Web[1], WebLINX[8]	跨网站任务构造、候选元素筛选、历史动作建模
网页评测环境	WebArena[2], VisualWebArena[3]	端到端成功率、视觉网页任务、执行式评测
真实网页 Agent	WebVoyager[4], SeeAct[5]	截图 + 结构信息 + VLM 的操作闭环，SoM grounding
实验框架	BrowserGym, AgentLab[9]	标准化观察空间、动作空间和批量实验管理
开源工程	browser-use[19], Skyvern[20]	浏览器工具调用、任务编排、日志记录和工程化接口
通用 GUI 评测	OSWorld[10], Windows Agent Arena[11], AndroidWorld[12]	真实系统环境、任务初始化、状态检查和跨平台操作视角
GUI 视觉解析	UI-TARS[13], OmniParser[14], SeeClick/ScreenSpot[15], ScreenSpot-Pro[16]	屏幕解析、视觉 grounding、坐标定位和长链 GUI 操作启发

第 3 章　系统方案设计

3.1 总体架构

本项目采用**混合观察输入**：网页截图保留视觉布局与图标信息，DOM/Accessibility Tree 保留可执行元素信息，Set-of-Marks 标注图将候选元素编号化，便于模型输出稳定动作。整体流程为：

Observe（观察）→ **Plan**（规划）→ **Act**（执行）→ **Check**（检查）→ **Verify**（验证）。

3.2 核心模块

浏览器执行层。使用 Playwright[18] 控制 Chromium，支持打开网页、点击、输入、滚动、等待、返回和截图。执行层只接受**结构化动作 JSON**，并限制域名、最

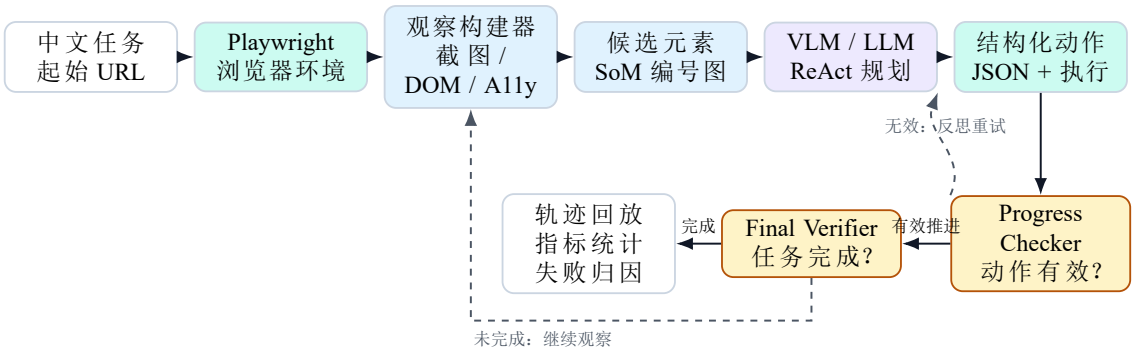


图 3.1　网页多模态 Agent 的 Observe–Plan–Act–Check–Verify 闭环架构

大步数和高风险操作，避免登录、支付、下单或提交敏感信息。

观察构建层。每一步提取页面截图、DOM、Accessibility Tree 和候选交互元素。候选元素包括按钮、链接、输入框、下拉框、菜单项等，并记录文本、角色、位置和可见性。随后生成 Set-of-Marks 标注截图，把候选元素编号叠加在页面上。

界面理解与动作规划层。 VLM 负责 OCR、视觉布局理解和目标元素定位；LLM/VLM 使用 ReAct[7] 风格提示词，结合任务目标、历史动作和当前观察输出下一步动作。动作空间包括 `click`、`type`、`scroll`、`press`、`back`、`wait`、`answer`、`done`。

进展检查与完成验证。Progress Checker 判断动作是否让页面更接近任务目标；若点击无响应、页面未变化、陷入循环或弹窗遮挡，则触发反思重试。Final Verifier 在 Agent 输出 `done` 后检查 URL、页面证据和答案字段，降低**“未完成但自信结束”**的风险。

3.3　模型选择

我们计划接入**多种 VLM/LLM 模型**进行对比评测。系统层面统一观察输入、动作 JSON 格式和浏览器执行接口，使不同模型在相同任务、相同候选元素标注和相同评测指标下运行。这样可以先用能力较强的 API 模型跑通完整闭环，也可以进一步接入本地开源模型，比较不同模型在任务成功率、平均步数、延迟、成本、动作有效性和失败类型上的差异。

实验实现上，我们将优先保证完整 Agent 闭环、轨迹记录和批量评测流程可运行；随后在相同测试集上比较 API 模型与 7B/8B 级开源 VLM/LLM 的表现。针对本地部署开源模型，我们计划租用单张 24GB 4090 GPU 进行量化推理，并记录显存占用、推理耗时和每个 episode 的平均延迟等；针对 API 模型的使用，则记录

表 3.1　多模型接入与对比评测方案

模块	拟接入方案	评测关注点
界面理解	Qwen2.5-VL / Qwen3-VL[17], CogAgent, InternVL 等开源 VLM，以及 API 多模态模型	截图理解、OCR、元素定位、SoM 编号选择准确率
动作规划	Qwen / DeepSeek / GPT 系列 LLM，或端到端 VLM Agent	ReAct[7] 推理稳定性、JSON 动作合法率、长链任务规划能力
浏览器控制	Playwright[18]	点击、输入、滚动、等待、截图、trace 记录与异常处理能力
标注增强	DOM 候选元素 + Accessibility Tree + SoM[6]	将坐标点击转化为元素编号选择，比较有无 SoM 时的成功率和定位失败率

调用次数、输入/输出 token 和估算费用等。

第 4 章　初步数据集与实验设想

4.1　难度分层

课程要求在同一网站上定义至少 3 个 task。我们计划将这 3 个 task 采取“难度分层”：每个网站尽量覆盖 easy、medium、hard 三档任务，分层依据主要包括任务步数、页面状态复杂度、是否需要跨页面记忆等等。

表 4.1　Easy / Medium / Hard 分层标准

难度	步数	能力要求	典型任务
Easy	3–15 步	目标明确，少量页面跳转；主要考察搜索、点击、滚动、元素定位和局部信息抽取	在 GitHub README 中找到安装命令或 quickstart；在 arXiv 摘要页定位 PDF、作者和 arXiv ID
Medium	20–80 步	多页面跳转、筛选、分页、跨页面比较和结构化抽取；需要保存中间状态并处理动态页面	比较多个开源 WebAgent 项目的许可证、部署方式和适用场景；围绕某个主题汇总 3–5 篇论文及其代码链接
Hard	100–300+ 步	长链任务、跨页面依赖、条件分支、局部回填、异常恢复和长期记忆；强调成本、延迟和稳定性	在受控仓库中读取 issue、定位相关文档/代码并生成修复方案或 patch 草稿；模拟 DS-160 类长表单填写流程

4.2　任务场景设计

任务来源上，我们打算参考 Mind2Web[1]、WebVoyager[4] 和 BrowserGym/AgentLab[9] 的任务组织方式：用真实网站、自然语言目标、起始 URL、成功判定和轨迹记录来定义 episode。由于真实在线网站会变化，我们不直接照搬 benchmark 的答案，而是复用其任务类型和评测思想，构建我们自己的小规模任务库。

我们初步计划构造 **6–10 个候选网站、约 30 个任务模板**。主实验阶段选择其

中至少 2 个网站，每个网站覆盖 easy/medium/hard 三类任务，每类任务至少 10 次独立测试，从而满足课程要求的 60 次以上测试；扩展阶段再增加 HuggingFace、MDN/Docs、Papers with Code 等网站，形成更丰富的跨站任务。

表 4.2　按难度组织的任务示例

网站/场景	Easy 示例	Medium 示例	Hard 示例
GitHub	在指定仓库 README 中找到安装命令、quickstart 或最小运行示例，并返回页面证据	比较 3–5 个 WebAgent 开源项目的许可证、依赖、部署方式和任务定位，输出结构化表格	在受控仓库中读取一个 issue，结合 README、release note 和相关文件，定位可能原因并生成修复建议或 patch 草稿
arXiv	打开指定论文摘要页或 PDF，返回标题、作者、arXiv ID 和分类信息	围绕一个主题检索 3–5 篇论文，比较时间、方法、数据集和是否提供代码链接	根据研究问题完成小型 literature survey：跨 arXiv、GitHub/Papers with Code 收集证据，整理相关工作脉络和可复现实验资源
HF / Docs	在模型卡或文档页找到安装、推理或 API 调用示例	比较多个模型卡的许可证、输入输出格式、硬件需求和适用任务	根据目标任务筛选模型、阅读模型卡限制，生成可执行的部署/调用方案并记录证据链接
长链表单	在普通表单页面完成少量字段填写和校验	在多页表单中根据给定资料完成分步填写、返回修改和字段一致性检查	在受控沙盒或演示模式下模拟 DS-160 类长表单流程，测试上百步操作中的记忆、回填和错误恢复能力

4.3　评测指标

表 4.3　评测指标体系

维度	指标	定义与作用
任务效果	任务成功率（SR）	成功完成任务数 / 总测试数；分别报告总体 SR 与 SR-Easy、SR-Medium、SR-Hard
用时	任务用时	从任务开始到结束的总耗时，并拆分为模型推理时间、浏览器等待时间和动作执行时间
成本	任务成本	若使用 API，记录每个 episode 的模型调用次数、输入/输出 token 与估算费用；若使用本地模型，记录 GPU 推理时间、显存占用和吞吐作为成本代理
效率	平均步数	每个 episode 的动作数量；长链 hard 任务单独报告 P50/P90 步数，避免均值掩盖极端案例
鲁棒性	纠错成功率（ESR）	触发反思或重试后最终完成任务的比例，衡量自我纠错机制价值
失败归因	Failure Taxonomy	将失败分为识别失败、定位失败、规划失败、执行失败、验证失败和网站因素，配合截图轨迹分析原因

上述指标按“**能否完成、完成代价、操作效率、失败可解释性**”组织。任务成功率是最核心的结果指标，并按 easy/medium/hard 分层报告；单任务用时衡量用户实际等待时间；单任务成本用于比较不同模型路线的调用费用或本地算力开销；平均步数反映规划路径是否简洁，尤其用于观察 hard 长链任务是否出现循环；纠

错成功率衡量反思与重试机制是否真正提升鲁棒性；失败归因则把每次失败映射到截图轨迹和动作序列，帮助后续定位是识别、定位、规划、执行还是验证环节出了问题。

4.4 时间计划与预期交付

表 4.4 项目计划

阶段	目标
第 9–10 周	完成调研、任务定义、系统方案和开题报告
第 10–11 周	跑通 Playwright 控制、截图、DOM/候选元素提取与 SoM 标注
第 11–12 周	接入 VLM/LLM，完成 ReAct 循环和 JSON 动作执行
第 12–13 周	实现 Progress Checker、Final Verifier 和轨迹日志系统
第 13–14 周	构造 6–10 个候选网站、约 30 个任务模板，并选取至少 2 个网站完成 60 次以上主测试
第 14–15 周	完成 baseline/消融实验，统计指标并整理失败类型
第 15–16 周	完成 Web UI、轨迹回放、Demo 视频和最终报告

预期交付包括：

- 一个可运行的网页多模态 Agent 原型，支持用户输入任务和起始 URL；
- 一套按 easy/medium/hard 组织的小规模任务库，覆盖 6–10 个候选网站和约 30 个任务模板；
- 至少 2 个真实网站、60 次以上独立测试的评测结果，包含成功率、用时、成本、平均步数、纠错成功率和失败归因；
- 关键模块或模型路线的对比结果，例如不同 VLM/LLM、是否启用 SoM 标注、是否启用纠错机制；
- 每个 episode 的截图序列、SoM 标注图、动作 JSON、模型调用记录和轨迹回放；
- 最终报告、代码仓库 README 和 3–5 分钟 Demo 视频。

4.5 预期创新点

1. **分层任务设计。**将课程要求的同站点多任务扩展为 easy/medium/hard 三档，覆盖从十步以内操作到百步级长链任务，能更清楚地展示 Agent 能力边界。
2. **面向真实可用性的评测体系。**不只报告成功率，还系统统计用时、成本、平均步数、纠错成功率和失败归因，使不同模型和模块配置能够横向比较。
3. **混合观察输入。**同时使用网页截图、DOM、Accessibility Tree 和 SoM 标注图，

兼顾视觉布局与可执行元素信息。

4. **元素编号式 grounding。**将“预测点击坐标”转化为“选择候选元素编号”，降低模型定位偏差。
5. **闭环纠错与完成验证。**通过 Progress Checker 判断动作是否有效推进任务，并在 Agent 声称完成后再次检查页面证据，减少循环、误操作和幻觉完成。
6. **轨迹级错误分析。**保存截图序列、动作序列和模型调用记录，对失败原因进行分类归因，形成可复盘、可改进的实验材料。

参 考 文 献

[1] Deng et al. Mind2Web: Towards a Generalist Agent for the Web. https://arxiv.org/abs/2306.06070

[2] Zhou et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. https://arxiv.org/abs/2307.13854

[3] Koh et al. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. https://arxiv.org/abs/2401.13649

[4] He et al. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. https://arxiv.org/abs/2401.13919

[5] Zheng et al. GPT-4V(ision) is a Generalist Web Agent, if Grounded. https://arxiv.org/abs/2401.01614

[6] Yang et al. Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V. https://arxiv.org/abs/2310.11441

[7] Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629

[8] Lù et al. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. https://arxiv.org/abs/2402.05930

[9] BrowserGym / AgentLab. https://github.com/ServiceNow/AgentLab

[10] Xie et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. https://arxiv.org/abs/2404.07972

[11] Bonatti et al. Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale. https://arxiv.org/abs/2409.08264

[12] Rawles et al. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. https://arxiv.org/abs/2405.14573

[13] Qin et al. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. https://arxiv.org/abs/2501.12326

[14] Lu et al. OmniParser for Pure Vision Based GUI Agent. https://arxiv.org/abs/2408.00203

[15] Cheng et al. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. https://arxiv.org/abs/2401.10935

[16] Li et al. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. https://arxiv.org/abs/2504.07981

[17] Qwen2.5-VL Technical Report. https://arxiv.org/abs/2502.13923

[18] Playwright Documentation. https://playwright.dev/python/docs/intro

[19] browser-use. https://github.com/browser-use/browser-use

[20] Skyvern. https://github.com/Skyvern-AI/skyvern